Experiment 09
Input Form Validation using HTML, CSS, JavaScript, and Validator.js

Aim

To design and develop a web-based input form that validates user data such as **email** and **phone number**, and displays appropriate error messages using **HTML, CSS, JavaScript, and Validator.js**.

Procedure

Step 1: Create HTML Structure

- Design a form with input fields for **email** and **phone number**
- Add labels, placeholders, and a submit button
- Include elements below each input field to display error messages

Step 2: Apply CSS Styling

- Style the form with a clean and centered layout
- Add padding, borders, and rounded corners

Step 3: Add Validator.js Library

- Include Validator.js using a CDN link
- This library helps in validating input fields easily

Step 4: Implement JavaScript Validation

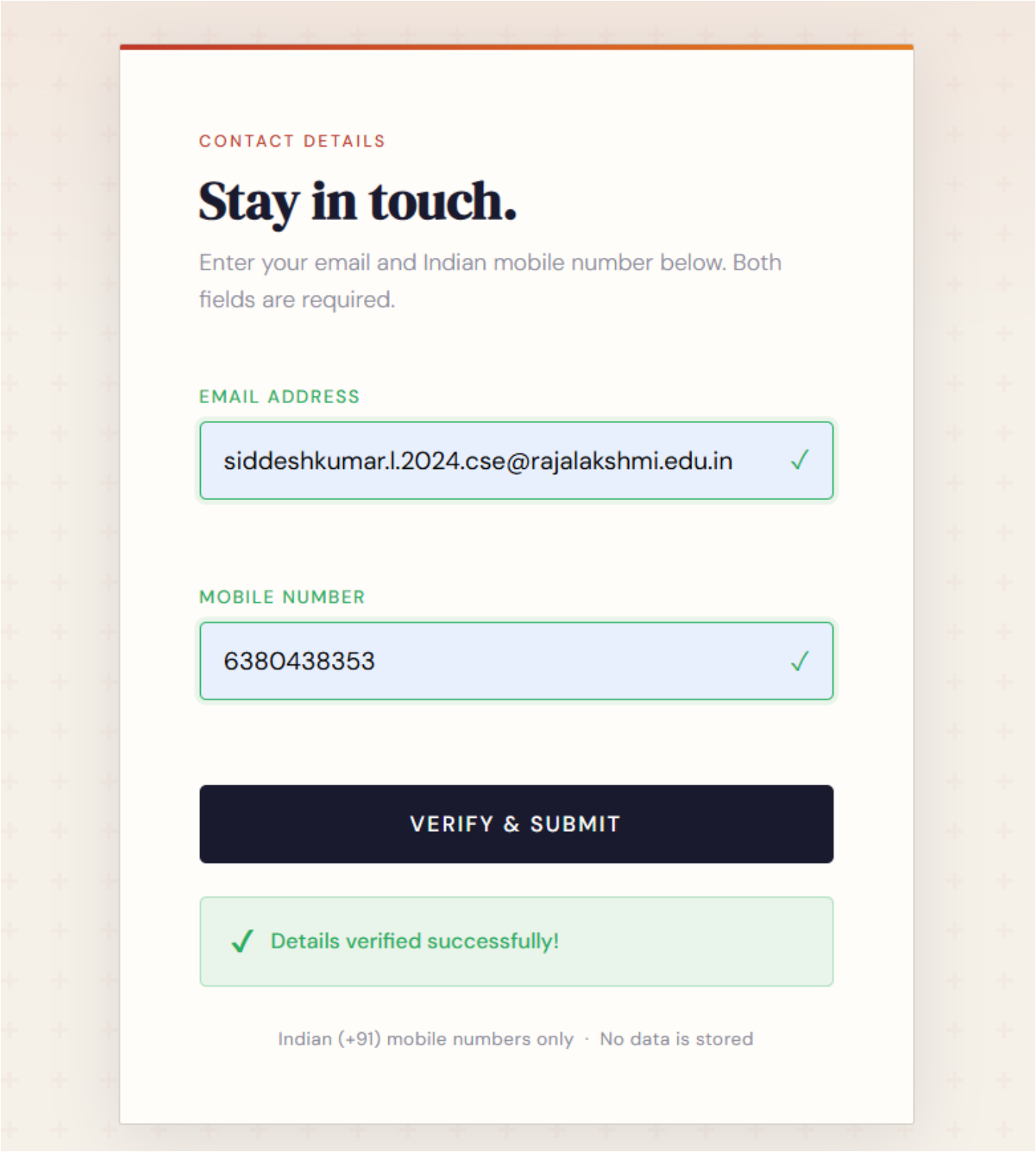
- Use validator.isEmail() to validate email
- Use validator.isMobilePhone() for phone validation (with "en-IN")
- Display error messages for invalid inputs
- Highlight valid and invalid fields
- Prevent form submission if validation fails

Step 5: Add User Interaction

- Validate inputs on **blur** and **input events**
- Show real-time feedback to the user
- Display a success message when all inputs are valid

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Contact Validator</title>

  <!-- ============================================
       EXTERNAL DEPENDENCIES
       Validator.js CDN for client-side validation
       Google Fonts for typography
  ============================================ -->
  <script src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.11.0/validator.min.js"></script>
  <link href="https://fonts.googleapis.com/css2?family=DM+Serif+Display&family=DM+Sans:wght@300;400;500&display=swap" re

  <style>
    /* ============================================
       CSS CUSTOM PROPERTIES (Design Tokens)
       Central palette and spacing system
    ============================================ */
    :root {
      --ink:        #1a1a2e;
      --paper:      #f5f0e8;
      --accent:     #c0392b;
      --valid:      #27ae60;
      --invalid:    #e74c3c;
      --muted:      #8a8a9a;
      --border:     #d4cfc6;
      --card-bg:    #fffdf9;
      --shadow:     0 8px 40px rgba(26,26,46,0.10);
      --radius:     4px;
      --transition: 0.22s cubic-bezier(0.4, 0, 0.2, 1);
    }
```

```html
    /* ============================================
       FORM SUBMIT HANDLER
       Run full validation; prevent default if invalid.
       Show success toast on a clean submission.
    ============================================ */
    form.addEventListener('submit', (e) => {
      e.preventDefault(); // Always intercept; let our logic decide

      // Mark both fields as touched so errors are shown even if untouched
      touched.email = true;
      touched.phone = true;

      const emailOk = validateEmail();
      const phoneOk = validatePhone();

      if (!emailOk || !phoneOk) {
        // Focus the first invalid field for accessibility
        if (!emailOk) emailInput.focus();
        else phoneInput.focus();
        return; // Stop — do not submit
      }

      // ---- All valid: show success feedback ----
      successToast.classList.add('visible');

      // In a real app you would do:
      // fetch('/api/contact', { method: 'POST', body: new FormData(form) })
      console.log('Form submitted:', {
        email: emailInput.value.trim(),
        phone: phoneInput.value.trim(),
      });
    });
  </script>
```



Result

The input form was successfully designed and implemented.

- Valid email and phone numbers are accepted and highlighted correctly
- Invalid inputs display appropriate error messages